SOBEY SCHOOL OF BUSINESS
MASTER OF BUSINESS ANALYTICS
ACCOUNTING & FINANCIAL ANALYTICS (MBAN5570)
ASSIGNMENT - 4

Instructor: Mohammad M. Rahaman
Lecture Location: SB415
Meeting Time: M-W: 1-4PM

Office: SB142
Phone: (902) 420 5730
E-mail: `Mohammad.Rahaman@smu.ca`

DUE DATE:April 15, 2026 (Brightspace Dropbox)

Please read the following instructions carefully: This is your final assignment. This assignment would be done using Python. I will give you all the relevant information here. Only one copy of your write-up should be submitted. Report your results clearly, and omit your computational details if they are not asked for. Explain briefly when necessary. Please do not bury your answers in large tables of numbers. Type all your answers (whether numerical or not) in a few pages. When you submit, make sure that you also give me your `.ipnyb` file. Do not hand in un-edited Python output. Submit at the designated Dropbox in Brightspace. You can download all relevant data files for this assignment from the Brightspace Course Page. The assignment will be graded out of 10 and all questions are equally weighted.

I. NLP and AI Intensity Analysis (Word Cloud + Text Analytics)

In this question, you will apply Natural Language Processing (NLP) techniques to analyze corporate communication. You can search for these transcripts on the company's website or on financial news websites. Make sure you refer to the links to your transcripts in your notebook. You can also use the following websites:

- https://seekingalpha.com/earnings/earnings-call-transcripts
- https://www.fool.com/earnings-call-transcripts/

- https://www.marketbeat.com/earnings/transcripts/

Step 1: Data Collection

- Collect earnings conference call transcripts for the past two years for:
  - Apple Inc.
  - Nvidia
  - ExxonMobil

Step 2: Text Processing

- Clean and preprocess the text (remove stopwords, punctuation, etc.)

Step 3: Word Cloud

- Create a word cloud using the top 100 most frequent words for each company

Step 4: AI Intensity Measure

- Define a set of AI-related keywords (e.g., AI, machine learning, neural networks, generative AI) and compute:

$$\text{AI Intensity} = \frac{\text{Number of AI-related sentences}}{\text{Total number of sentences}}$$

Step 5: Sentiment Analysis

- Perform sentiment analysis on the conference call transcripts using an NLP library (e.g., TextBlob)
- Compute an overall sentiment score (e.g., polarity) for each company
- Optionally, compare sentiment across time (e.g., quarterly trends)

Step 6: Analysis

- Compare AI intensity and sentiment across the three firms

- Comment on:
  - Which firms are at the **center of the AI revolution**
  - Which firms are at the **periphery**
- Support your answer using word clouds, AI intensity measures, and sentiment analysis

Please note the following: It is important to note that this exercise may require some additional research and reference to the tutorials provided in the course. Additionally, the final exercise should be submitted in a report format that includes detailed explanations of each step, as well as any code and visualizations used. You should use Google Colab notebook to complete this exercise, as it provides an efficient and accessible way to write and run code in a browser-based environment. You should include text blocks and inline comments where appropriate to provide context and explanation for your code, as well as to describe your findings and observations throughout the exercise.

- Other Resources:
  - TextBlob documentation: https://textblob.readthedocs.io/en/dev/
  - Pandas documentation: https://pandas.pydata.org/docs/
  - Matplotlib documentation: https://matplotlib.org/stable/contents.html

II. ML Application to Predicting Stock Prices using ARIMA

In this exercise, you will use `ARIMA` (Autoregressive Integrated Moving Average) to predict the adjusted closing price of the stock of your choice. Please use the yfinance library to extract the stock price data, and then split the data into training and testing sets. Finally, use ARIMA to train the model on the training set, test it on the testing set, and use it to make predictions on future stock prices. We have discussed various time-series models in the class. The ARIMA (Autoregressive Integrated Moving Average) is a popular model used for time series forecasting. It combines Autoregressive (AR) and Moving Average (MA) to model the time series data, and includes a first-difference term to account for non-stationarity in the data.

The `auto_arima` is an automated version of `ARIMA` that uses a grid search algorithm to automatically select the optimal values for the ARIMA parameters. This makes it easier to use and less dependent on user expertise, as it handles the selection of the optimal parameters automatically. To predict time series data for stock prices, you can use `ARIMA` or `auto_arima` to fit a model to the historical data and then use this model to make predictions for future time points. We typically split the historical data into training and testing sets, with the training data used to fit the model and the testing data used to evaluate its performance. Once the model has been trained, we can use it to make predictions for future time points by feeding in new data points to the model. The performance of the model can be evaluated by comparing the predicted values to the actual values.

The first step is to install the required libraries. Please use `yfinance` to extract the stock price data (pick a reasonable time period) and `ARIMA` to build the model. You can install these libraries using the following commands:

```
!pip install yfinance
!pip install pmdarima
```

Once you have installed the required libraries, you need to import them into your program. We will also import other necessary libraries such as pandas and numpy.

```
import yfinance as yf
import pandas as pd
import numpy as np
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
import pmdarima as pm
from datetime import datetime
```

The first step in building your predictive model is to collect historical stock data. There are several sources where you can get this data, but for this exercise, please use the yfinance library to get the historical stock prices of your stock. Before you can train the model, you need to preprocess the data. Please start by extracting the adjusted closing price data from the stock data and converting it into a pandas series.

```
stock_price = df[‘Adj Close’]
stock_price_series = pd.Series(stock_price)
```

Next, split the data into training and testing sets. It is customary to use the first 80% of the data for training and the remaining 20% for testing.

```
from sklearn.model_selection import train_test_split
train_data, test_data = train_test_split(stock_price_series, test_size=0.2, shuffle=False)
```

Now that you have preprocessed the data, you can build the `ARIMA` model. You have to use the `pmdarima` library to automatically select the optimal values for the `ARIMA` parameters.

```
model = pm.auto_arima(train_data, seasonal=False, suppress_warnings=True)
```

This above code uses the `auto_arima` function from the pmdarima library to automatically select the optimal values for the ARIMA parameters (p, d, q) for a given time series train_data, where `p` is the number autoregressive lags, `q` is the number of Moving Average lags, and `d` is degree of referencing. The resulting model will be used to predict values for the test data, and the Root Mean Square Error (RMSE) value is calculated to evaluate model performance. The seasonal=False parameter specifies that the model does not have any seasonal component, and suppress_warnings=True suppresses any warnings that may arise during the model fitting process. Now that you have built the model, you can train it on the training data using the `fit()` function.

```
model.fit(train_data)
```

Once you have trained the model on the training data, you can now test it on the testing data to see how well it performs.

```
predictions = model.predict(n_periods=len(test_data))
```

To evaluate the performance of the model, you have to calculate the Root Mean Squared Error (RMSE) between the predicted values and the actual values. The RMSE tells how much the model fits the training data.

```
rmse = np.sqrt(mean_squared_error(test_data, predictions))
print('RMSE:', rmse)
```

After you have trained and tested your `ARIMA` model, please use python to visualize the predictions of the model by plotting the predicted values against the actual values. Are you satisfied with your model? Once you are satisfied with our model, please predict a single value from a given date input. In this exercise, I want you to predict the Adjusted Closing price of your stock for a date that does not exist in your dataset. Based on your short-term prediction,what would be your recommendation: Sell, Buy, Hold?

Please note the following: It is important to note that this exercise may require some additional research and reference to the tutorials provided in the course. Additionally, the final exercise should be submitted in a report format that includes detailed explanations of each step, as well as any code and visualizations used. You should use Google Colab notebook to complete this exercise, as it provides an efficient and accessible way to write and run code in a browser-based environment. You should include text blocks and inline comments where appropriate to provide context and explanation for their code, as well as to describe their findings and observations throughout the exercise.

III. ML and Credit Ratings

For this question, please load the dataset `ML_CreditRating.csv`, which gives you more than 3000 credit ratings of companies (identified as `id`). The dataset also gives you a company's cash holding (Cash/Total Assest), liquidity (Current Assets/Current Liability), and interest coverage ratio (EBITDA/Interest Expense). I want you to build a Machine Learning Model of Credit Ratings. The inputs of the model are cash holding, liquidity (current ratio), and interest coverage ratio. The out of the model is credit ratings such as AAA, AA, A, BBB, BB+ etc.

Please split the data into training and testing sample. Make sure that you have 80% of the inputs and outputs are allocated for training and the rest of the 20% of inputs and outputs are allocated to testing. Please note that your input are all numeric variables while your output is a string variable. First, identify an appropriate model, i.e., linear regression, decision tree, neural network, or random forest etc.) and then train your data using the model. Next, evaluate the performance of your model fit using the testing sample. If you are satisfied with the performance of your model fit, predict the credit rating of a company with 10% cash holding, 2.5 Current Ratio, and 8.5 Interest Coverage Ratio